

Component-based Software Engineering

Soon Phei Tin

Topic

- Part 1: Introduction to CBSE
- Part 2: Components in Spring Boot: Foundations
- Part 3: Component Interfaces, Composition, and Lifecycle
- Part 4: Advanced Component Features in Spring Boot
- Part 5: Component-Based Architecture with Spring Boot
- Part 6: Challenges, Best Practices, and Wrap-Up

Part 1: Introduction to CBSE

- CBSE is the software engineering practices that developing software mainly based on software components.
- We will focus on three core principles: **Components, Interfaces, and Composition**
- Beyond the core principles, understanding CBSE requires a deeper look at **components, component models, and middleware**, as well as how they interrelate.

Benefits and Challenges

Benefits

- Reusability reduces development time and costs.
- Modularity enhances maintainability and scalability.
- Easier testing and debugging of isolated components.

Challenges

- Dependency management and version conflicts.
- Overhead from inter-component communication.
- Designing components that are both reusable and specific enough for practical use.

Components

- In CBSE, a component is a modular, reusable, and independently deployable unit of software that encapsulates specific functionality and data.
- It provides services to other parts of the system through well-defined interfaces, hiding its internal implementation details.
- This modularity promotes reusability and maintainability.

Components

Here are the summarized characteristics of components:

- **Self-contained:** Each component encapsulates its own logic and data.
- **Reusable:** Designed to be used across different systems or contexts.
- **Replaceable:** Can be swapped with another component that provides the same interface.
- **Interface-driven:** Interact with other components solely through defined interfaces, promoting loose coupling and high cohesion.

Components in Spring Boot

- In Spring Boot, components are typically classes annotated with `@Component`, `@Service`, `@Repository`, or `@Controller`.
- These annotations tell the Spring Inversion of Control (IoC) container to manage the class, handling its instantiation, dependency injection, and lifecycle
- UserService is a self-contained component that provides a specific service (fetching user data).
- Other parts of the application can use it without knowing how it retrieves the data, aligning with CBSE's emphasis on encapsulation and modularity.

```
@Service
public class UserService {
    public User getUserById(Long id) {
        // Logic to fetch a user by ID
        return new User(id, "John Doe");
    }
}
```

Interfaces

- Interfaces in CBSE define the contract that components must follow.
- They specify what services a component provides (and sometimes requires) without exposing its internal workings.
- This abstraction enables loose coupling, as components can be swapped out as long as they adhere to the same interface.

Interfaces in Spring Boot

- In Spring, interfaces are commonly used to define service contracts, with concrete implementations injected at runtime.
- The UserRepository interface acts as a contract, allowing different implementations (e.g., JPA, JDBC, or in-memory) to be used interchangeably.

```
public interface UserRepository {  
    User findById(Long id);  
}
```

- This reflects CBSE's focus on well-defined interfaces for component interaction.

```
@Repository  
public class JpaUserRepository implements UserRepository {  
    @Override  
    public User findById(Long id) {  
        // JPA-specific implementation to fetch user from database  
        return new User(id, "Jane Doe");  
    }  
}
```

Composition

- Composition in CBSE involves assembling components into a complete system by connecting them through their interfaces.
- This process leverages dependency relationships to create a functional whole, ensuring that components work together seamlessly.

Composition in Spring Boot

- Spring Boot uses dependency injection to compose components.
- The UserService is composed with a UserRepository via constructor inject
- Spring's IoC container manages this composition, connecting the components at runtime based on their interfaces.

```
//  
@Service  
public class UserService {  
    private final UserRepository userRepository;  
  
    @Autowired  
    public UserService(UserRepository userRepository) {  
        this.userRepository = userRepository;  
    }  
  
    public User getUserById(Long id) {  
        return userRepository.findById(id);  
    }  
}
```

Component Models

A component model is a framework or set of standards that governs how components are defined, implemented, and composed.

It provides:

- **Conventions:** Rules for creating components (e.g., how to specify interfaces).
- **Interaction Mechanisms:** How components communicate (e.g., method calls, events).
- **Lifecycle Management:** How components are instantiated, used, and destroyed.

Component Models

- Examples of component models include
 - Enterprise JavaBeans (EJB)
 - Microsoft's COM
 - Spring
- A component model ensures consistency and interoperability across a system.

Spring Component Model

Relies on Plain Old Java Objects (POJOs) enhanced with annotations:

- `@Component` (and its derivatives) identifies components.
- `@Autowired` specifies dependencies.
- Spring Boot's auto-configuration further simplifies this by automatically setting up components based on classpath dependencies.

Component and Component Models

- The component model defines the “rules of the game” for components.
- Components are instances that conform to these rules, ensuring they can be managed and composed effectively within the system.

Middleware

Middleware is software that sits between the operating system and applications, providing services that simplify component interaction and system integration.

It handles:

- **Communication:** Enabling components to talk to each other, even across networks (e.g., via RPC or messaging).
- **Data Management:** Abstracting database access or caching.
- **Infrastructure Services:** Offering security, transaction management, or load balancing.

Middleware

In CBSE, middleware acts as the “glue” that supports component composition and execution, especially in distributed systems.

Spring Example:

- **Spring Data:** Acts as middleware by providing a consistent data access layer between components and databases.
- **Spring Cloud:** Offers middleware features like service discovery (Eureka), load balancing (Ribbon), and circuit breaking (Hystrix) for distributed Spring Boot applications.
- For instance, a Spring Boot app might use Spring Cloud to connect a UserService component on one server with a UserRepository component on another, abstracting the network complexity.

Components, Component Models, and Middleware

- **Components and Component Models:** The component model defines how components are structured and interact. Components are the concrete realizations of this model.
- **Components and Middleware:** Middleware provides the runtime environment and services that components rely on to function. A component like UserService might use middleware (e.g., Spring Data) to access a database without handling the low-level details itself.

Components, Component Models, and Middleware

- **Component Models and Middleware:** The component model often assumes a specific middleware environment. Spring's model, for instance, integrates tightly with its middleware (e.g., Spring Cloud), ensuring components can leverage distributed system features seamlessly.

Together, they form a layered system:

1. **Components** deliver customized functionality.
2. **Component Models** standardize their creation and composition.
3. **Middleware** enables their execution and interaction, especially in complex or distributed scenarios.

Part 2: Components in Spring Boot: Foundations

Objectives

- Learn how Spring Boot implements CBSE principles.
- Understand component definition and basic dependency injection.

Spring Boot Overview

Spring Boot is an extension of the Spring Framework that streamlines the development of production-ready applications by reducing configuration complexity and providing sensible defaults.

Key features include:

- **Auto-configuration:** Spring Boot automatically configures components based on the dependencies present in the classpath.
- **Embedded Servers:** Built-in support for web servers like Tomcat or Jetty allows applications to run without external server setup.
- **Opinionated Defaults:** Provides pre-configured settings for common tasks, which developers can override as needed.

Spring Boot for CBSE

Spring Boot's architecture is inherently component-based, making it an ideal framework for implementing CBSE principles:

- Components are managed by the Spring IoC container, ensuring modularity and reusability.
- Dependency injection facilitates loose coupling between components.
- Auto-configuration and starter dependencies simplify the integration and composition of components into cohesive applications.

Components in Spring Boot

A **component** in Spring is any class that is managed by the Spring IoC container. These classes are typically annotated to indicate their role and enable automatic detection by Spring.

- **Core Annotations:**

- **@Component:** Spring-managed bean.
- **@Service:** Encapsulates business logic.
- **@Repository:** Data access component.
- **@Controller / @RestController:** Components for HTTP requests handling.

- **Component Registration:**

Spring Boot uses a process called **component scanning** to automatically detect and register components. By default, Spring scans the package containing the main application class and all its sub-packages.

Components in Spring Boot

- Example `import org.springframework.stereotype.Service;`

```
@Service
public class GreetingService {
    public String greet() {
        return "Hello, World!";
    }
}
```

- The `@Service` annotation marks `GreetingService` as a component.
- Spring Boot detects it during component scanning and registers it as a bean in the IoC container.

Inversion of Control (IoC) Container

The Spring's component management system, embodying the **IoC** principle.

In IoC, the framework takes control of creating and managing objects, rather than the application code doing so manually.

- **Role of the IoC Container:**

- Instantiates components (referred to as **beans**).
- Configures them by injecting dependencies and setting properties.
- Manages their lifecycle from creation to destruction.
- Stores beans in the **application context**, which serves as the runtime environment.

Dependency Injection (DI)

- **Dependency Injection** is a design pattern where a component's dependencies are provided by the framework rather than being instantiated within the component itself.
- This enhances modularity, testability, and maintainability.
- **Types of Dependency Injection:**
 - **Constructor Injection:** Recommended for mandatory dependencies.
 - **Setter Injection:** Suitable for optional dependencies or runtime changes.
 - **Field Injection:** Direct injection into fields (less preferred due to testability concerns)

Constructor Injection

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class OrderService {
    private final PaymentService paymentService;

    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Setter Injection

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class NotificationService {
    private EmailService emailService;

    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

Field Injection

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
}
```

Advantages of DI

- **Advantages of DI:**

- **Decoupling:** Components are independent of how their dependencies are created.
- **Testability:** Dependencies can be mocked or stubbed in tests.
- **Flexibility:** Dependencies can be swapped or reconfigured without altering component code.

Component Discovery and Customization

- Spring Boot's components are discovered during component scanning, but it also offers customization options for fine-grained control.
- **Component Scanning:** By default, Spring scans the package of the main application class and its sub-packages.
- Customize this behavior with `@ComponentScan`:

```
@SpringBootApplication
@ComponentScan(basePackages = {"com.example.services", "com.example.repositories"})
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

Component Discovery and Customization

- Define beans explicitly in a configuration class using @Bean
- This approach is useful for third-party classes or when more control is needed

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import com.zaxxer.hikari.HikariDataSource;

@Configuration
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        return new HikariDataSource();
    }
}
```


Practical Example: Building a Simple REST API

```
public class Product {  
    private Long id;  
    private String name;  
    private double price;  
  
    public Product(Long id, String name, double price) {  
        this.id = id;  
        this.name = name;  
        this.price = price;  
    }  
  
    // Getters and setters  
    public Long getId() { return id; }  
    public void setId(Long id) { this.id = id; }  
    public String getName() { return name; }  
    public void setName(String name) { this.name = name; }  
    public double getPrice() { return price; }  
    public void setPrice(double price) { this.price = price; }  
}
```

Practical Example: Building a Simple REST API

```
import org.springframework.stereotype.Service;
import java.util.Arrays;
import java.util.List;

@Service
public class ProductService {
    public List<Product> getAllProducts() {
        // Hardcoded for simplicity
        return Arrays.asList(
            new Product(1L, "Laptop", 999.99),
            new Product(2L, "Smartphone", 499.99)
        );
    }
}
```

Practical Example: Building a Simple REST API

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {
    private final ProductService productService;

    @Autowired
    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @GetMapping
    public List<Product> getProducts() {
        return productService.getAllProducts();
    }
}
```

Practical Example: Building a Simple REST API

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ProductApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductApplication.class, args);
    }
}
```

Part 3: Component Interfaces, Composition, and Lifecycle

Objectives

- Understand the use of interfaces for component abstraction.
- Explore component composition techniques and lifecycle management.

Interfaces in CBSE and Spring Boot

Interfaces are a cornerstone of CBSE because they establish a **contract** that components must follow.

This contract specifies what services a component offers (its capabilities) and what it requires (its dependencies).

Interfaces in CBSE and Spring Boot

By defining interactions through interfaces rather than concrete implementations, interfaces enable:

- **Polymorphism:** Different implementations of the same interface can be swapped seamlessly.
- **Loose Coupling:** Components depend on abstractions, reducing tight dependencies on specific implementations.
- **Testability:** Interfaces simplify mocking dependencies during unit testing.

Spring Boot Example: Defining a Service Interface

- The `PaymentProcessor` interface defines a contract for processing payments.
- `CreditCardProcessor` and `PayPalProcessor` provide concrete implementations that can be used interchangeably.

```
public interface PaymentProcessor {  
    void processPayment(double amount);  
}  
  
@Service  
public class CreditCardProcessor implements PaymentProcessor {  
    @Override  
    public void processPayment(double amount) {  
        System.out.println("Processing credit card payment of $" + amount);  
    }  
}  
  
@Service  
public class PayPalProcessor implements PaymentProcessor {  
    @Override  
    public void processPayment(double amount) {  
        System.out.println("Processing PayPal payment of $" + amount);  
    }  
}
```


Benefits of Using Interfaces

- **Flexibility:** Switch between payment processors (e.g., credit card to PayPal) by injecting a different implementation.
- **Extensibility:** Add new payment methods (e.g., CryptoProcessor) without altering existing code.
- **Testability:** Mock the PaymentProcessor interface in tests to isolate dependent components.

Composition Techniques

- In CBSE, **composition** refers to assembling components into a functional system by connecting them through their interfaces.
- Spring Boot achieves this primarily through **dependency injection (DI)**, a mechanism where the framework provides a component's dependencies automatically.
- Spring supports several DI techniques, each suited to different scenarios.

Best Practice for Dependency Injection

- **Prefer Constructor Injection:** Use it for mandatory dependencies to ensure immutability and explicitness.
- **Use Setter Injection Sparingly:** Reserve it for optional or reconfigurable dependencies.
- **Best Practice for Dependency Injection** Opt for constructor or setter injection to improve testability and maintainability.

Component Lifecycle Management

- Spring Boot's Inversion of Control (IoC) container manages the entire lifecycle of components (beans), from instantiation to destruction.
- Understanding this lifecycle allows developers to perform initialization tasks (e.g., loading resources) and cleanup tasks (e.g., closing connections) at the appropriate stages.

Key Phases of the Component Lifecycle

- **Creation**

- The IoC container creates the bean instance.
- Dependencies are injected based on the chosen DI method (constructor, setter, or field).

- **Initialization**

- After dependency injection, Spring invokes initialization logic.
- **@PostConstruct**: Annotate a method to run it after the bean is fully constructed.

- **Usage**

- The bean is fully initialized and available for use by other components or services.

- **Destruction**

- Before the application context closes, Spring invokes destruction logic.
- **@PreDestroy**: Annotate a method to run it before the bean is destroyed.

```
@Service
public class CacheService {
    private Map<String, Object> cache = new HashMap<>();

    @PostConstruct
    public void init() {
        System.out.println("CacheService initialized. Loading cache...");
        // Simulate loading initial cache data
        cache.put("defaultKey", "defaultValue");
    }

    public void put(String key, Object value) {
        cache.put(key, value);
    }

    public Object get(String key) {
        return cache.get(key);
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("CacheService shutting down. Clearing cache...");
        cache.clear();
    }
}
```

Why Lifecycle Management Matters

- **Resource Initialization:** Set up connections, load configurations, or pre-populate data (e.g., cache).
- **Resource Cleanup:** Release resources like database connections, file handles, or memory to prevent leaks.
- **Controlled Behavior:** Ensure components start and stop gracefully, maintaining application stability.

Best Practices for Component Design

- Define Interfaces for Abstraction
- Favor Constructor Injection
- Single Responsibility Principle
- Leverage Lifecycle Hooks (@PostConstruct and @PreDestroy)
- Avoid Circular Dependencies
- Document Interfaces (e.g., Javadoc)

Practical Example: Composing Components with Interfaces

Scenario

- An OrderService depends on a PaymentProcessor (interface with multiple implementations) and a NotificationService. The system processes payments and sends notifications when an order is placed.

1. Payment processor interface

```
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

Practical Example: Composing Components with Interface

- Implement concrete payment processors

```
@Service
public class CreditCardProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}
```

```
@Service
public class PayPalProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}
```

Practical Example: Composing Components with Interface

- Define NotificationService

```
@Service
public class NotificationService {
    public void sendNotification(String message) {
        System.out.println("Sending notification: " + message);
    }
}
```

Practical Example: Composing Components with Interface

- Compose the OrderService with dependencies

```
@Service
public class OrderService {
    private final PaymentProcessor paymentProcessor;
    private final NotificationService notificationService;

    @Autowired
    public OrderService(PaymentProcessor paymentProcessor, NotificationService
notificationService) {
        this.paymentProcessor = paymentProcessor;
        this.notificationService = notificationService;
    }

    public void placeOrder(double amount, String customerEmail) {
        paymentProcessor.processPayment(amount);
        notificationService.sendNotification("Order placed successfully for " +
customerEmail);
    }
}
```

Practical Example: Composing Components with Interface

- Resolve multiple implementations
- Option 1: @Primary

```
@Service
@Primary
public class CreditCardProcessor implements PaymentProcessor {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}
```

Practical Example: Composing Components with Interface

- Resolve multiple implementations
- Option 2: @Qualifier

```
@Service
public class OrderService {
    private final PaymentProcessor paymentProcessor;
    private final NotificationService notificationService;

    @Autowired
    public OrderService(@Qualifier("creditCardProcessor") PaymentProcessor
paymentProcessor,
                        NotificationService notificationService) {
        this.paymentProcessor = paymentProcessor;
        this.notificationService = notificationService;
    }

    // ... rest of the code
}
```

Part 4: Advanced Component Features in Spring Boot

- **Objective:**

- Explore advanced Spring Boot component management techniques, including custom scopes, lazy loading, dynamic dependency injection, and conditional bean creation
- Design flexible and scalable applications aligned with Component-Based Software Engineering (CBSE) principles.

- **Prerequisites:** Complete hands-on exercises 1 and 2

Overview

This section introduces advanced Spring Boot features that enhance component flexibility and adaptability:

- **Component Scopes with Real-World Scenarios**
- **Advanced Dependency Resolution**
- **Programmatic Bean Definition with Conditions**

These concepts enable you to tackle real-world challenges like circular dependencies, runtime configuration, and dynamic component selection, reinforcing CBSE's emphasis on modularity and reusability.

Component Scopes with Real-World Scenarios

Spring Boot manages component lifecycles through **scopes**, determining how instances are created and shared. Advanced scopes and custom implementations offer fine-grained control.

- **Standard Scopes:**

- **Singleton:** One instance per ApplicationContext (default).
- **Prototype:** New instance per request.

- **Web-Specific Scopes:**

- **Request:** One instance per HTTP request.
- **Session:** One instance per user session.
- **Application:** One instance per ServletContext (web app lifecycle).

- **Custom Scopes:** Define bespoke scopes for specific use cases (e.g., thread-local or transaction-specific).

Example: Request and Session Scopes

- Check the *sample code* to see how @RequestScope and @SessionScope are used

Request Scopes

- Practical Scenario 1: Tracking Request-Specific Metrics
 - In an e-commerce API, you want to track metrics for each incoming request (e.g., processing time, number of database queries) without persisting this data beyond the request.
- Practical Scenario 2: Temporary User Input Validation
 - A user submits a complex form (e.g., product customization options) via a POST request, and you need to validate and process it within a single request.

Example: Request and Session Scopes

Session scopes

- Practical Scenario 1: Shopping Cart Management
 - In the e-commerce API, users add items to a shopping cart across multiple requests before checking out.
- Practical Scenario 2: Multi-Step User Registration Wizard
 - A user completes a multi-step registration process (e.g., personal info, shipping address, payment details) across several requests.

Advanced Dependency Resolution

- Complex applications require sophisticated dependency management.
- Real-world systems often involve intricate relationships between components, dynamic runtime conditions, and the need for flexibility, scalability, and maintainability.
- As applications grow in complexity advanced techniques like @Lazy, ObjectProvider, @Qualifier, and programmatic bean definitions are necessary.

Why Sophisticated Dependency Management is Needed

- **Ambiguity in Dependency Resolution:** When multiple beans implement the same interface
- **Circular Dependencies:** In tightly coupled designs, two beans may depend on each other, causing Spring to fail during context initialization.
- **Dynamic or Conditional Dependencies:** Applications may need to select dependencies at runtime based on configuration, environment, or user input
- **Performance Optimization:** Eager initialization of all beans can slow startup in large applications

Why Sophisticated Dependency Management is Needed

- **Scalability and Flexibility:** As systems scale (e.g., microservices or modular designs), hardcoding dependencies with `@Autowired` limits adaptability to new requirements or implementations
- **Testing and Maintenance:** Basic `@Autowired` tightly couples components, making it harder to mock or replace dependencies in tests or during maintenance.

Ambiguity in Dependency Resolution

- When multiple beans implement the same interface, we use `@Primary` or `@Qualifier` to specify the required implementation.
- Now let explore how `ObjectProvider` enhance more flexibility

```
public interface PaymentProcessor {  
    void process(double amount);  
}  
  
@Service  
@Qualifier("creditCard")  
public class CreditCardProcessor implements PaymentProcessor {  
    @Override  
    public void process(double amount) {  
        System.out.println("Processing $" + amount + " via Credit Card");  
    }  
}  
  
@Service  
@Qualifier("paypal")  
public class PayPalProcessor implements PaymentProcessor {  
    @Override  
    public void process(double amount) {  
        System.out.println("Processing $" + amount + " via PayPal");  
    }  
}
```

```

@Service
public class DynamicOrderService {
    private final ObjectProvider<PaymentProcessor> processors;

    public DynamicOrderService(ObjectProvider<PaymentProcessor> processors) {
        this.processors = processors;
    }

    public void placeOrder(double amount, String paymentType) {
        PaymentProcessor processor = processors.stream()
            .filter(p ->
                p.getClass().getSimpleName().toLowerCase().contains(paymentType.toLowerCase()))
            .findFirst()
            .orElseThrow(() -> new IllegalArgumentException("No processor for " +
                paymentType));
        processor.process(amount);
    }
}

```

```

@RestController
@RequestMapping("/orders")
public class OrderController {
    private final DynamicOrderService orderService;

    public OrderController(DynamicOrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    public String createOrder(@RequestParam double amount, @RequestParam String
        paymentType) {
        orderService.placeOrder(amount, paymentType);
        return "Order placed";
    }
}

```


Circular Dependencies

- In the e-commerce API, ProductService needs InventoryService to check stock, and InventoryService needs ProductService to fetch product details for stock updates.

```
@Service
public class ProductService {
    private final Lazy<InventoryService> inventoryService;

    @Autowired
    public ProductService(Lazy<InventoryService> inventoryService) {
        this.inventoryService = inventoryService;
    }

    public Product getProduct(Long id) {
        Product product = new Product(id, "Laptop", 999.99, 10); // Dummy data
        int stock = inventoryService.get().checkStock(id);
        product.setStock(stock);
        return product;
    }
}
```

```
@Service
public class InventoryService {
    private final ProductService productService;

    @Autowired
    public InventoryService(ProductService productService) {
        this.productService = productService;
    }

    public int checkStock(Long productId) {
        Product product = productService.getProduct(productId);
        return product.getStock(); // Simplified logic
    }
}
```

Circular Dependencies

- When you mark a dependency as lazy, Spring does not inject the actual target bean during the startup phase. Instead, it injects a proxy object.

Step	Action	ProductService	InventoryService
1	Spring starts creating ProductService	Instantiating...	Not Started
2	Spring sees ProductService needs @Lazy InventoryService. Spring creates a proxy and hands it to ProductService.	Created	Not Started
3	Spring starts creating InventoryService	Created	Instantiating...
4	Spring sees InventoryService needs ProductService. Spring finds the created ProductService in cache and injects it to InventoryService.	Created	Created

Circular Dependencies

- Compare to the use of InventoryService without @Lazy

Step	Action	Status
1	Spring try to initiate ProductService	Pending...
2	It sees ProductService needs InventoryService. It pauses ProductService and looks for InventoryService	Searching...
3	Spring try to initiate InventoryService	Pending...
4	It sees InventoryService needs ProductService. It pauses InventoryService and looks for ProductService	Searching...
5	Spring realizes ProductService is still "In Creation"	Crash

Programmatic Bean Definition with Conditions

Programmatic bean definition allows dynamic control over component creation, enhanced by conditions and lifecycle customization.

Programmatic bean definition in Spring Boot refers to the explicit creation and configuration of beans using Java code, typically via `@Configuration` classes and `@Bean` annotations

- **@Configuration and @Bean:** Define beans explicitly.
- **@Conditional:** Create beans based on runtime conditions.
- **BeanPostProcessor:** Customize bean creation globally.

Why We Need Programmatic Bean Definition

- **Fine-Grained Control Over Bean Creation:** Automatic scanning creates beans based on annotations with limited customization options.
 - **Example:** Configuring a third-party object with specific settings.
- **Dynamic or Conditional Bean Creation:** Beans may need to be created based on runtime conditions
 - **Example:** Loading different implementations based on a property file.
- **Integration with Third-Party Libraries:** Many third-party libraries don't come with Spring annotations, so you need to wrap them in Spring-managed beans manually.
 - **Example:** Integrating a database connection pool like HikariCP.

Example: Conditional Bean Definition

- **Scenario:** Enable a logging service only in production

```
@Configuration
public class AppConfig {
    @Bean
    @ConditionalOnProperty(name = "app.environment", havingValue = "prod")
    public LoggingService loggingService() {
        return new LoggingService();
    }
}

public class LoggingService {
    @PostConstruct
    public void init() {
        System.out.println("LoggingService initialized");
    }
}
```

Example: Configuring a Third-Party Library Bean

- In the e-commerce API, you need to integrate a third-party library (e.g., a payment gateway SDK like Stripe) that doesn't use Spring annotations.

```
@Configuration
public class PaymentConfig {
    @Bean
    public RequestOptions stripeRequestOptions() {
        Stripe.apiKey = "sk_test_your_key"; // External config
        properties)
        return RequestOptions.builder()
            .setConnectTimeout(30_000) // Custom timeout
            .setReadTimeout(80_000)
            .build();
    }
}
```

```
@Service
public class PaymentService {
    private final RequestOptions stripeOptions;

    public PaymentService(RequestOptions stripeOptions) {
        this.stripeOptions = stripeOptions;
    }

    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " with Stripe options: " +
            stripeOptions);
    }
}
```

Part 5: Component-Based Architecture with Spring Boot

- **Objective:**
 - Explore advanced architectural patterns and techniques in Spring Boot, including Domain-Driven Design (DDD), and hexagonal (ports and adapters) architecture to create a robust, modular e-commerce system aligned with Component-Based Software Engineering (CBSE) principles.

Overview

This section equips you with the tools to architect complex applications:

- **Modular Design with DDD:** Structure the application using entities, aggregates, and bounded contexts.
- **Hexagonal Architecture:** Implement a ports and adapters pattern for flexibility and testability.

Modular Design with Domain-Driven Design (DDD)

DDD is a design approach that aligns software structure with business domains, promoting modularity and clarity. Key concepts include:

- **Entities:** Objects with identity (e.g., Product).
- **Aggregates:** Clusters of entities with a root (e.g., Order with OrderItems).
- **Bounded Contexts:** Separate domains with clear boundaries (e.g., Product vs. Order).

Entity

- An **entity** is an object in the domain model that has a distinct identity and a lifecycle
- entities are defined by a unique identifier (e.g., an ID) that persists across changes to their state
- Entities model the “things” in the domain that need to be tracked individually
- They are the building blocks of the domain model, ensuring that the system reflects real-world concepts with unique identities.

```
import lombok.Data;
```

✕ Collapse

≡ Unwrap



```
@Data
```

```
public class Product {
```

```
    private Long id;           // Unique identifier
```

```
    private String name;
```

```
    private double price;
```

```
    private int stock;
```

```
    public Product(Long id, String name, double price, int stock) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.price = price;
```

```
        this.stock = stock;
```

```
    }
```

```
    // Business logic
```

```
    public void reduceStock(int quantity) {
```

```
        if (quantity > stock) {
```

```
            throw new IllegalStateException("Insufficient stock for product " +  
id);
```

```
        }
```

```
        stock -= quantity;
```

```
    }
```

```
    public void updatePrice(double newPrice) {
```

```
        if (newPrice < 0) {
```

```
            throw new IllegalArgumentException("Price cannot be negative");
```

```
        }
```

```
        this.price = newPrice;
```

```
    }
```

```
}
```

Aggregate

- An **aggregate** is a cluster of related entities and value objects that are treated as a single unit for data consistency and transactional boundaries.
- Each aggregate has a root entity, called the **aggregate root**, which serves as the entry point for interacting with the aggregate.
- Order Aggregate in E-Commerce
 - An Order aggregate includes the Order entity (root) and a collection of OrderItem entities. The total amount must reflect the items, and stock must be reserved atomically.

Order aggregate

```
@Data
@Entity
public class Order {
    @Id
    private Long id;           // Aggregate root identity
    @OneToMany(cascade = CascadeType.ALL, mappedBy = "order")
    private List<OrderItem> orderItems = new ArrayList<>();
    private double totalAmount;
    private String status;

    public Order() {}

    public void addItem(Long productId, int quantity, double unitPrice) {
        OrderItem item = new OrderItem(productId, quantity, unitPrice, this);
        orderItems.add(item);
        calculateTotal();
    }

    // Invariant enforcement
    public void calculateTotal() {
        totalAmount = orderItems.stream()
            .mapToDouble(item -> item.getQuantity() * item.getUnitPrice())
            .sum();
    }
}
```

```
@Data
@Entity
public class OrderItem {
    @Id
    @GeneratedValue
    private Long id;
    private Long productId;
    private int quantity;
    private double unitPrice;
    @ManyToOne
    @JoinColumn(name = "order_id")
    private Order order;

    public OrderItem() {}

    public OrderItem(Long productId, int quantity, double unitPrice, Order order) {
        this.productId = productId;
        this.quantity = quantity;
        this.unitPrice = unitPrice;
        this.order = order;
    }
}
```

Bounded Context

- A **bounded context** is a specific boundary within which a domain model is defined and applicable.
- It's a way to divide a large, complex domain into smaller, manageable parts, each with its own model, language, and rules.
- Bounded contexts manage complexity by splitting a large domain into focused subdomains, each with a clear purpose.
- ProductCatalog and OrderManagement Bounded Contexts
 - In the e-commerce API, ProductCatalog manages product listings, while OrderManagement handles order processing.

Bounded Context

Each bounded context with its own:

- **Domain Model:** A set of entities, aggregates, and value objects tailored to the context.
 - **Ubiquitous Language:** A shared vocabulary used by developers, domain experts, and the code itself within that context.
 - **Rules and Assumptions:** Context-specific business logic and constraints.
-
- Bounded contexts prevent the domain model from becoming a monolithic, overly generalized mess by allowing different parts of the system to have distinct interpretations of similar concepts (e.g., “Product”) without conflict.

Bounded Contexts in the E-Commerce

Example - ProductCatalog

- The ProductCatalog bounded context is responsible for managing the inventory of products available for sale.
- **Responsibilities:**
 - Maintaining product details (e.g., name, price, stock).
 - Managing stock levels (e.g., reducing stock when reserved).
 - Providing product information to other contexts (e.g., for ordering).

Bounded Contexts in the E-Commerce

Example - ProductCatalog

Ubiquitous Language

- **Product:** An item in the inventory with stock available for sale.
- **Stock:** The quantity of a product available for purchase.
- **Reduce Stock:** Reserve stock for an order, ensuring availability.
- **Restock:** Add more units to the inventory from a supplier.

Boundaries

- **What's Included:** Product definitions, stock management, and catalog-related operations.
- **What's Excluded:** Order processing, customer details, payment handling.

Bounded Contexts in the E-Commerce

Example - OrderManagement

- The OrderManagement bounded context handles the creation, processing, and tracking of customer orders.
- **Responsibilities:**
 - Creating and managing orders.
 - Calculating order totals based on product prices at the time of ordering.
 - Tracking order status (e.g., pending, confirmed, shipped).

Bounded Contexts in the E-Commerce

Example - OrderManagement

Ubiquitous Language

- **Order:** A customer's request to purchase products.
- **Order Item:** A line item in an order, tied to a product and quantity.
- **Total Amount:** The calculated cost of the order based on ordered items.
- **Confirm Order:** Finalize the order after validation.

Boundaries

- **What's Included:** Order creation, total calculation, status tracking.
- **What's Excluded:** Stock management, product catalog updates.

Benefit of Bounded Context

- **Clarity:** “Product” has a clear meaning in each context, avoiding confusion (e.g., stock in ProductCatalog vs. price snapshot in OrderManagement).
- **Modularity:** Each context can evolve independently (e.g., adding supplier logic to ProductCatalog doesn’t affect OrderManagement).
- **Reduced Coupling:** No shared database or model; contexts communicate via well-defined interfaces or events.
- **Scalability:** ProductCatalog and OrderManagement can be deployed separately, aligning with microservices

Challenges of Bounded Context

- **Context Mapping:** Define how ProductCatalog and OrderManagement relate (e.g., client/supplier relationship where OrderManagement depends on ProductCatalog).
 - Other possible mapping pattern: Anticorruption Layer, Published Language with Event-Driven Integration
- **Duplication:** Some data (e.g., product name) might be duplicated (e.g., in OrderItem for display), but this is intentional to keep contexts independent.
- **Integration Overhead:** REST calls or message queues add complexity (e.g., latency, error handling).

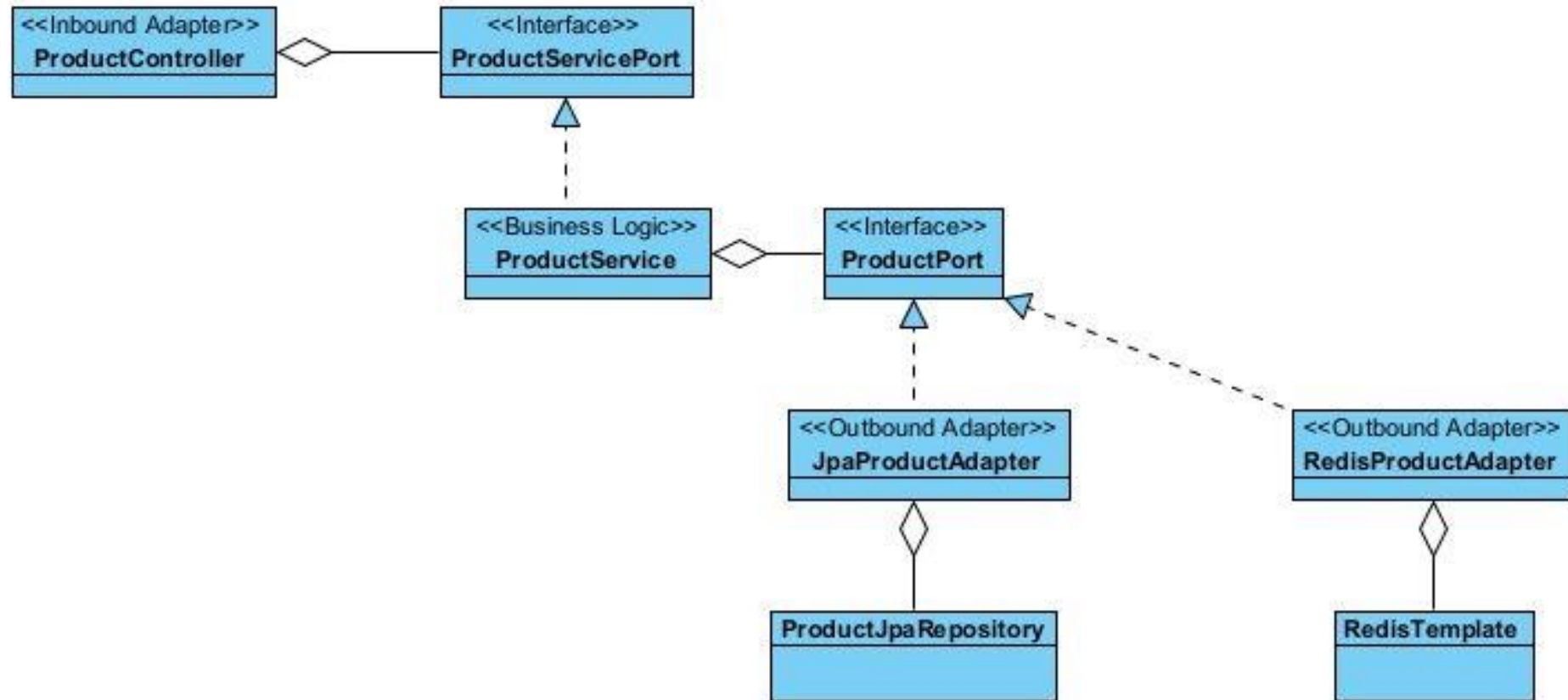
Hexagonal Architecture (Ports and Adapters)

- The **hexagonal architecture**, introduced by Alistair Cockburn
- Architectural style that decouples an application's business logic (the core) from its external systems (e.g., databases, user interfaces, APIs).
- It achieves this by defining:
 - **Ports**: Interfaces that specify what the core needs or provides.
 - **Adapters**: Concrete implementations that connect ports to external systems.

Hexagonal Architecture

- The application's business logic (the “hexagon”) is at the center, surrounded by ports and adapters.
- External systems interact with the core only through these well-defined boundaries.
- **Ports:** Abstract interfaces that define:
 - **Inbound Ports:** Services the core provides to the outside (e.g., API endpoints).
 - **Outbound Ports:** Services the core requires from the outside (e.g., data persistence).
- **Adapters:** Concrete classes that implement ports, bridging the core to external systems (e.g., REST controllers, JPA repositories).

Hexagonal Architecture



```
package com.example.productservice.port;

import com.example.productservice.core.Product;

public interface ProductPort {
    Product findById(Long id);
    void save(Product product);
}
```

```
package com.example.productservice.port;

import com.example.productservice.core.Product;

public interface ProductServicePort {
    Product updateStock(Long id, int quantity);
}
```

```

package com.example.productservice.core;

import com.example.productservice.port.ProductPort;
import org.springframework.stereotype.Service;

@Service
public class ProductService implements ProductServicePort {
    private final ProductPort productPort;

    public ProductService(ProductPort productPort) {
        this.productPort = productPort;
    }

    @Override
    public Product updateStock(Long id, int quantity) {
        Product product = productPort.findById(id);
        product.reduceStock(quantity);
        productPort.save(product);
        return product;
    }
}

```

```

import com.example.productservice.core.Product;
import com.example.productservice.port.ProductPort;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public class JpaProductAdapter implements ProductPort {
    private final ProductJpaRepository repository;

    public JpaProductAdapter(ProductJpaRepository repository) {
        this.repository = repository;
    }

    @Override
    public Product findById(Long id) {
        return repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("Product not found: " +
id));
    }

    @Override
    public void save(Product product) {
        repository.save(product);
    }
}

@Repository
interface ProductJpaRepository extends JpaRepository<Product, Long> {
}

```

```
package com.example.productservice.adapter;

import com.example.productservice.core.Product;
import com.example.productservice.core.ProductService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/products")
public class ProductController {
    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    @PutMapping("/{id}/stock")
    public Product updateStock(@PathVariable Long id, @RequestParam int quantity) {
        return productService.updateStock(id, quantity);
    }
}
```

Providing Additional Implementation

```
import com.example.productservice.core.Product;
import com.example.productservice.port.ProductPort;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.stereotype.Component;

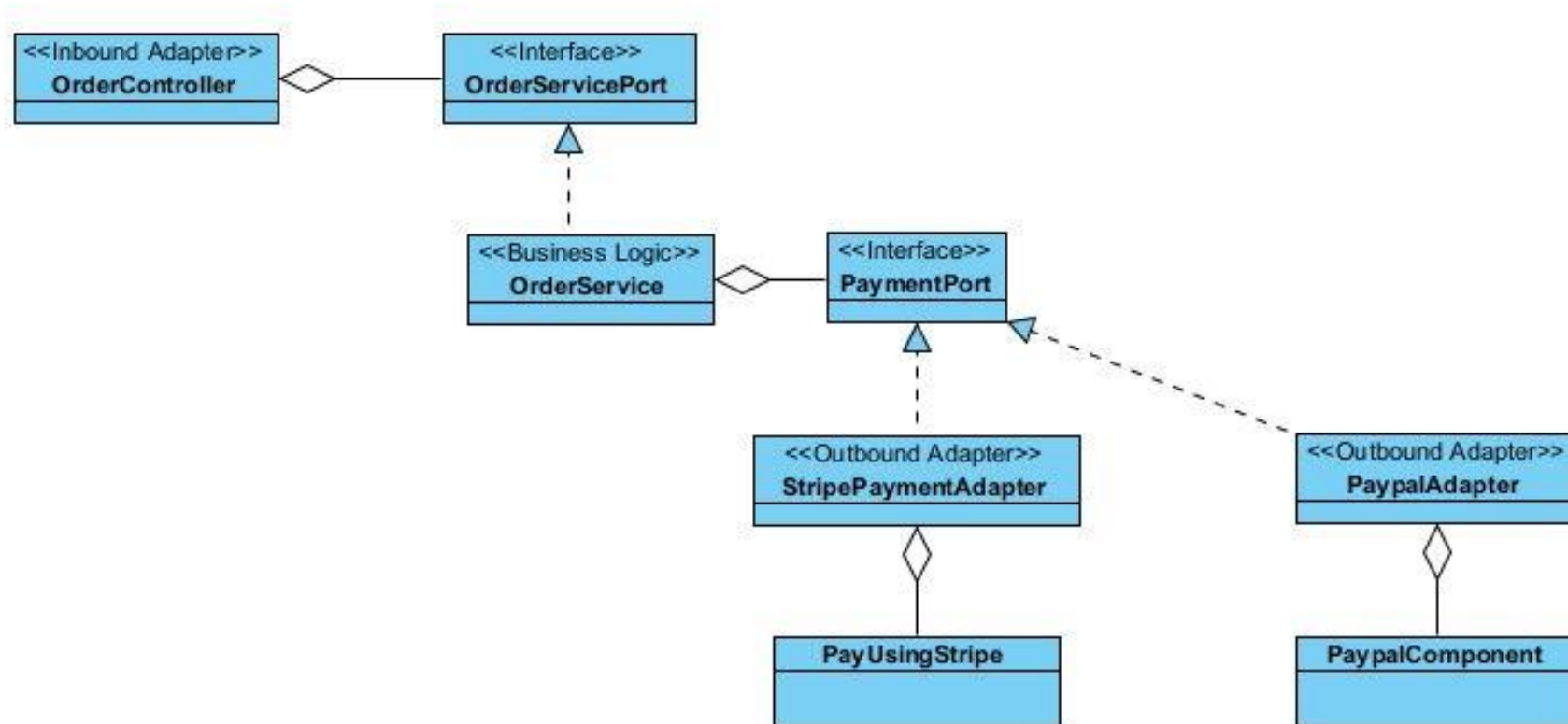
@Component
public class RedisProductAdapter implements ProductPort {
    private final RedisTemplate<String, Product> redisTemplate;

    public RedisProductAdapter(RedisTemplate<String, Product> redisTemplate) {
        this.redisTemplate = redisTemplate;
    }

    @Override
    public Product findById(Long id) {
        Product product = redisTemplate.opsForValue().get("product:" + id);
        if (product == null) {
            throw new IllegalArgumentException("Product not found: " + id);
        }
        return product;
    }

    @Override
    public void save(Product product) {
        redisTemplate.opsForValue().set("product:" + product.getId(), product);
    }
}
```

Payment Example



Benefits

- **Decoupling:** Business logic is independent of infrastructure (e.g., swap databases without changing the core).
- **Testability:** Mock adapters to test the core in isolation.
- **Flexibility:** Add new adapters (e.g., switch from JPA to MongoDB) without altering business logic.

Final Words

- We have seen how CBSE improved software development
 - Modularity
 - Reusability
 - Flexibility
 - Maintainability
- We have explored the CBSE in Spring Boot's way
 - Define components including Bean
 - IoC, DI
 - Flexibility of having multiple implementation
 - Use of middleware such as JPA and Spring Security
 - DDD and Hexagonal pattern